



THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tāmaki Makaurau
NEW ZEALAND

ADAPTER PATTERN

COMPSCI 230 OBJECT-ORIENTED SOFTWARE DEVELOPMENT

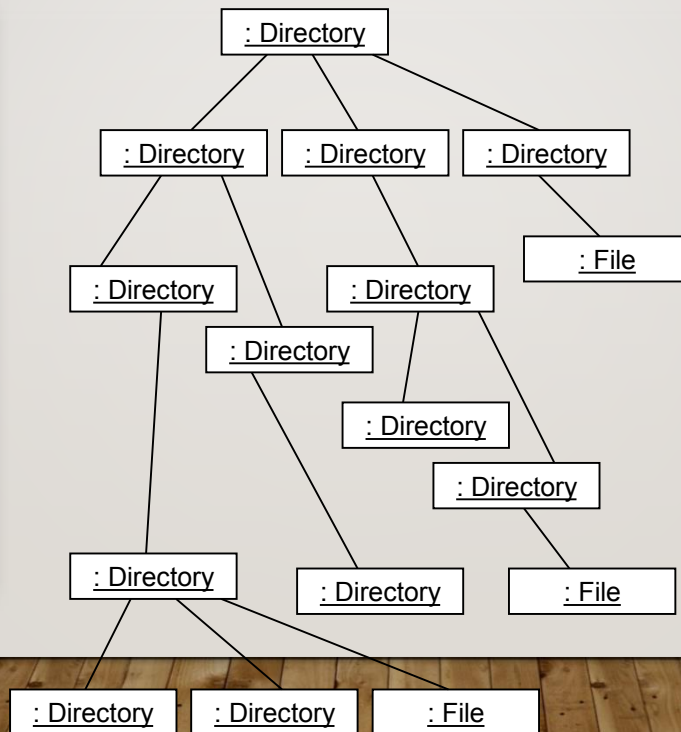
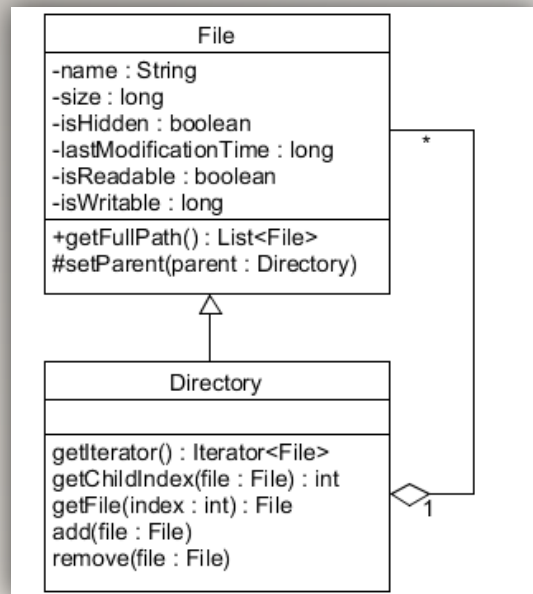
PARAMVIR SINGH



AGENDA

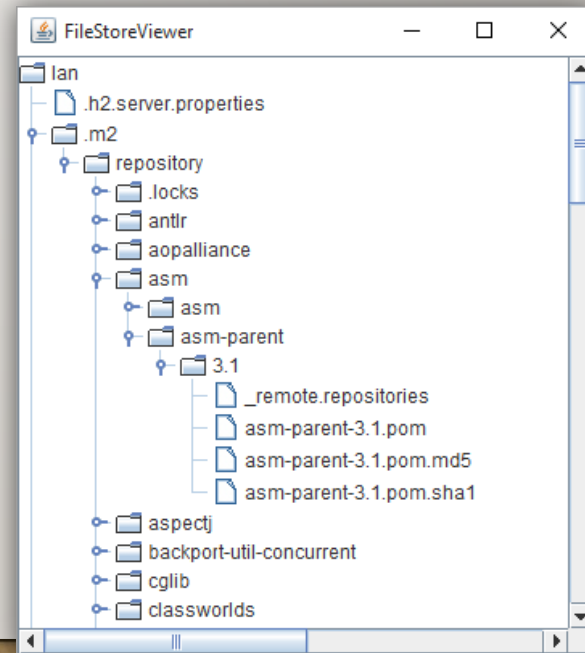
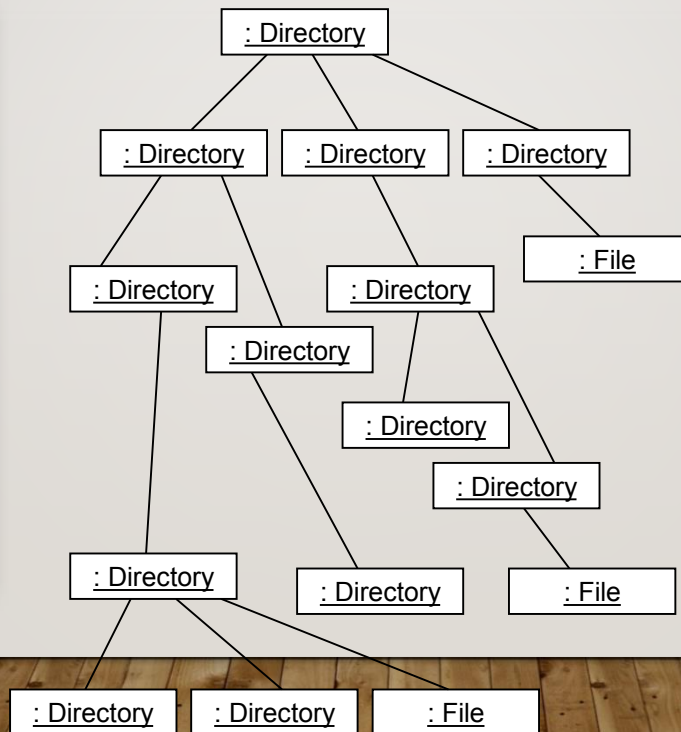
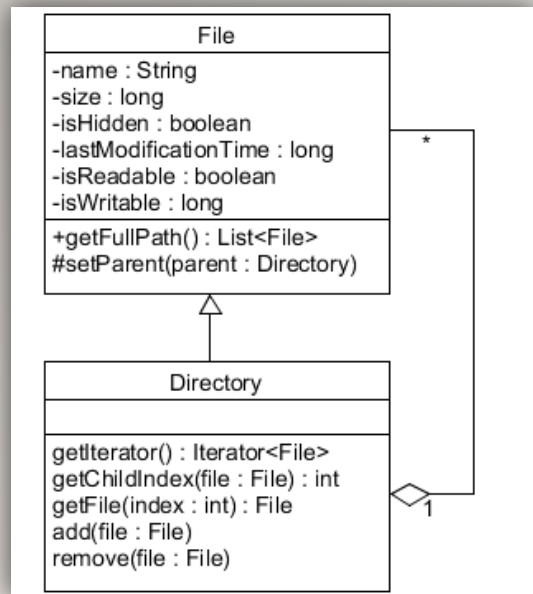
- Towards a Visual representation of FileStore Case
 - Reusing JTree and TreeModel
 - Critiquing three designs
- Description
- Solution
- Application
- Structure
- Containment vs Inheritance
- Code resources
 - File Store System

PREVIOUSLY ON COMPSCI 230

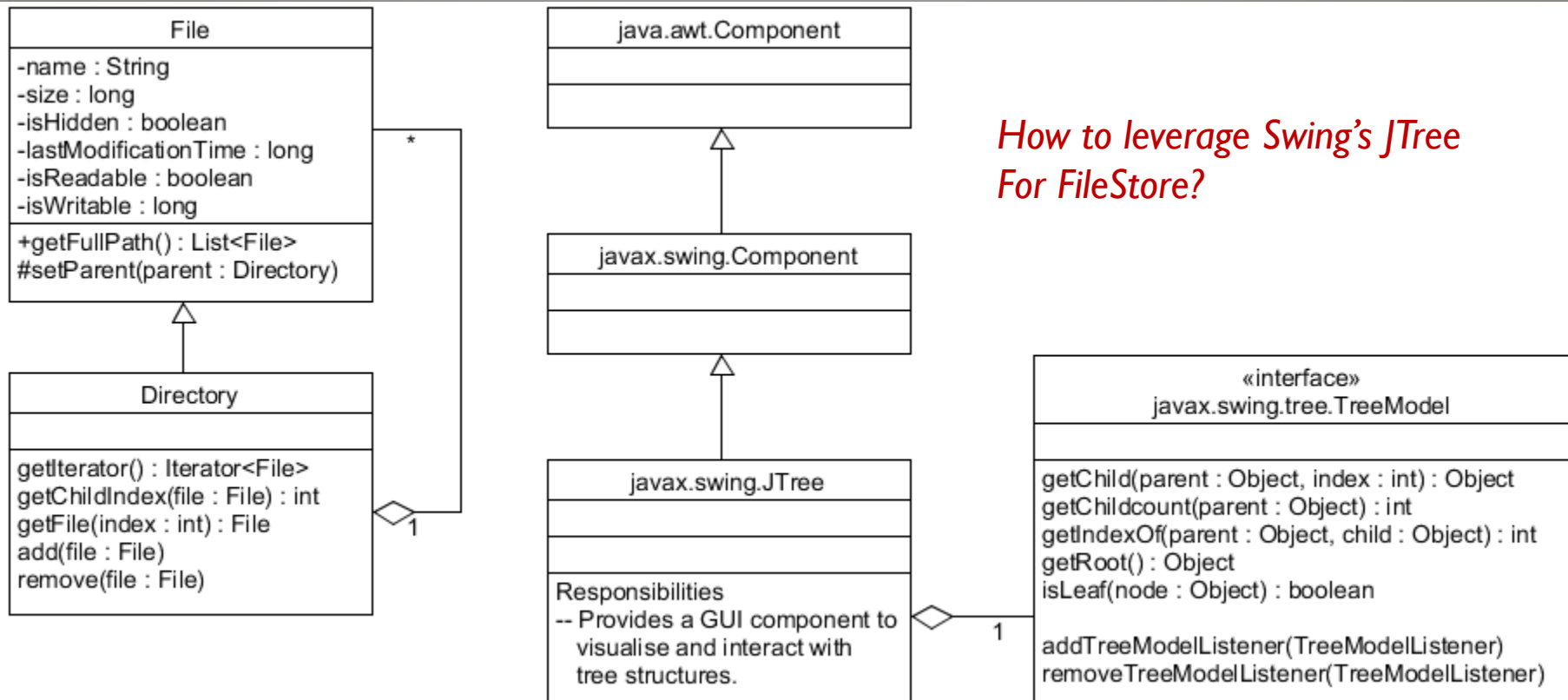


PREVIOUSLY ON COMPSCI 230

FILESTORE – A VISUAL REPRESENTATION?

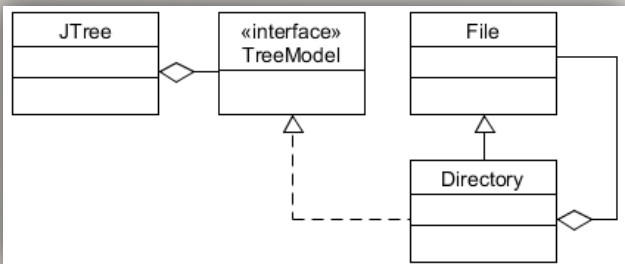


JAVA API CLASSES AVAILABLE FOR REUSE

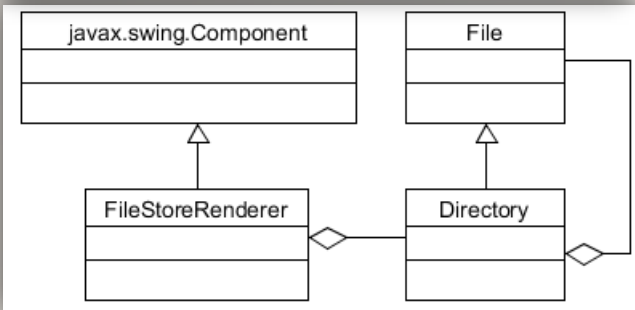


CRITIQUE THE THREE DESIGNS FOR THE PROBLEM

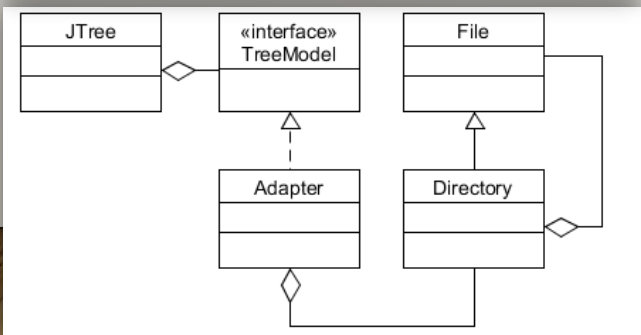
1.



2.



3.



Relative pros and cons for each design:

Design 1 – **edit** class **Directory** to make it implement **TreeModel**.

- In a production environment, making a change to an existing piece of software will cause that unit of software to go through a round of quality assurance.
- This option creates an undesirable coupling between class **Directory** and Java's Swing framework.
- This option is not possible if the source code for **Directory** is unavailable.

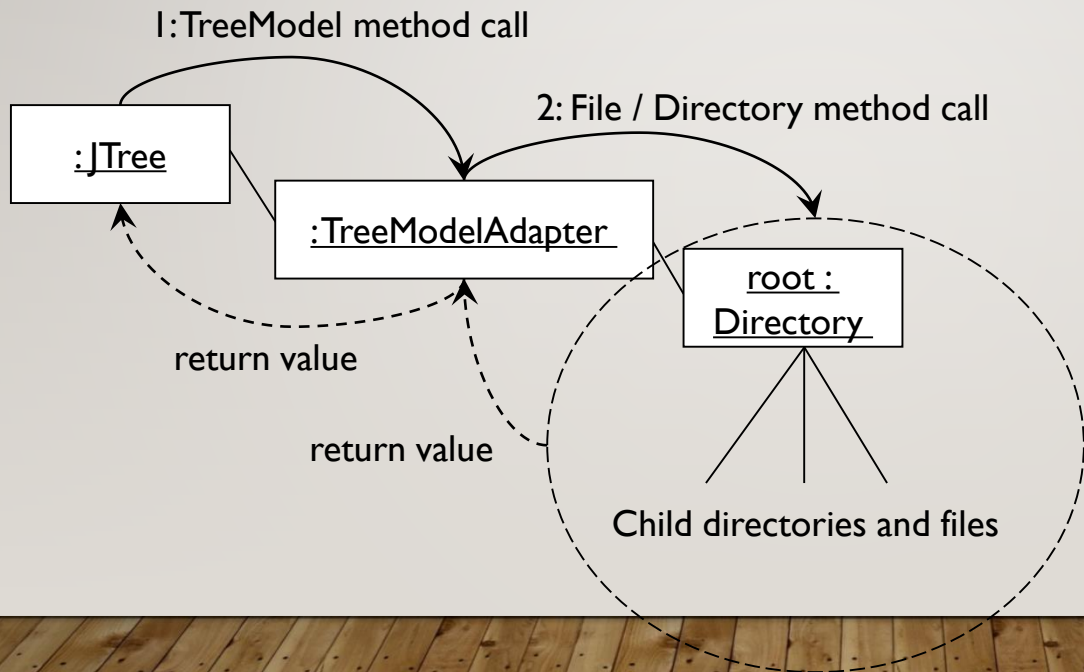
Design 2– write a new class, similar to **JTree**, that is dedicated to **rendering** the structure of a set of **File/Directory** objects.

- This is reinventing the wheel. There is no reuse of Swing's tried-and-tested **JTree** component; the new **FileStoreRenderer** class has very limited reuse.

Design 3 – introduce an **adapter class** (note: these have nothing to do with Java AWT's "adapter" class).

- No changes need be made to any existing code.
- Both **JTree** and the **File/Directory** classes are reused.

ADAPTERS: ANALOGY TO APPLICATION



ADAPTER CODE

```

public class TreeModelAdapter implements TreeModel {
    private Directory adaptee;

    public TreeModelAdapter( Directory root ) {
        adaptee = root;
    }

    public Object getChild() {
        return adaptee;
    }

    public int getChildCount( Object parent ) {
        int result = 0;

        if ( parent instanceof Directory ) {
            Directory directory = ( Directory ) parent;
            result = directory.getNumberOfChildren();
        }
        return result;
    }

    public boolean isLeaf( Object node ) {
        return ! ( node instanceof Directory );
    }
}

```

```

public Object getChild( Object parent, int index ) {
    if ( parent instanceof Directory ) {
        Directory dir = ( Directory ) parent;
        return dir.getFile( index );
    }
    return null;
}

public int getIndexOfChild( Object parent, Object child ) {
    int indexOfChild = -1;

    if ( parent instanceof Directory ) {
        Directory dir = ( Directory ) parent;
        Iterator<File> i = dir.getIterator();
        boolean found = false;
        int index = 0;
        while ( ! found && i.hasNext() ) {
            File current = i.next();
            if ( child == current ) {
                found = true;
                indexOfChild = index;
            } else
                index++;
        }
    }
    return indexOfChild;
}
}

```

ADAPTER CODE

```
public class FileStoreViewer extends JFrame {  
  
    public FileStoreViewer() {  
        super( "FileStoreViewer" );  
        Directory root = FileFactory.makeFileStore( );  
        TreeModel model = new TreeModelAdapter( root );  
  
        JTree view = new JTree( model );  
  
        JScrollPane treeScrollPane = new JScrollPane( view );  
        add( treeScrollPane );  
    }  
  
    public static void main( String[ ] args ) {  
        FileStoreViewer frame = new FileStoreViewer( );  
        frame.pack( );  
        frame.setVisible( true );  
    }  
}
```

Create a configuration of Directory and File objects representing the contents of your home directory.

Create the adapter, passing a reference to the root Directory object.

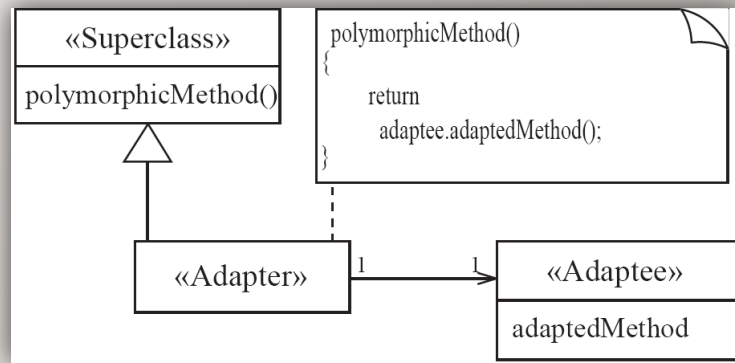
Create the JTree and introduce it to our adapter. The JTree will call the TreeModel methods on the adapter to gather the data necessary to render the Directory/File configuration.

Decorate the JTree component with scroll bars.

PATTERN DESCRIPTION

- **Context:**
 - You want the classes with incompatible interfaces work together
 - You are building an inheritance hierarchy and want to incorporate it into an existing class.
 - The reused class is also often already part of its own inheritance hierarchy.
- **Problem:**
 - How to convert the interface of a class into another interface clients expect.
 - How to obtain the power of polymorphism when reusing a class whose methods.
 - have the same function
 - but *not* the same signature
 - as the other methods in the hierarchy?
- **Forces:**
 - You do not have access to multiple inheritance or you do not want to use it.

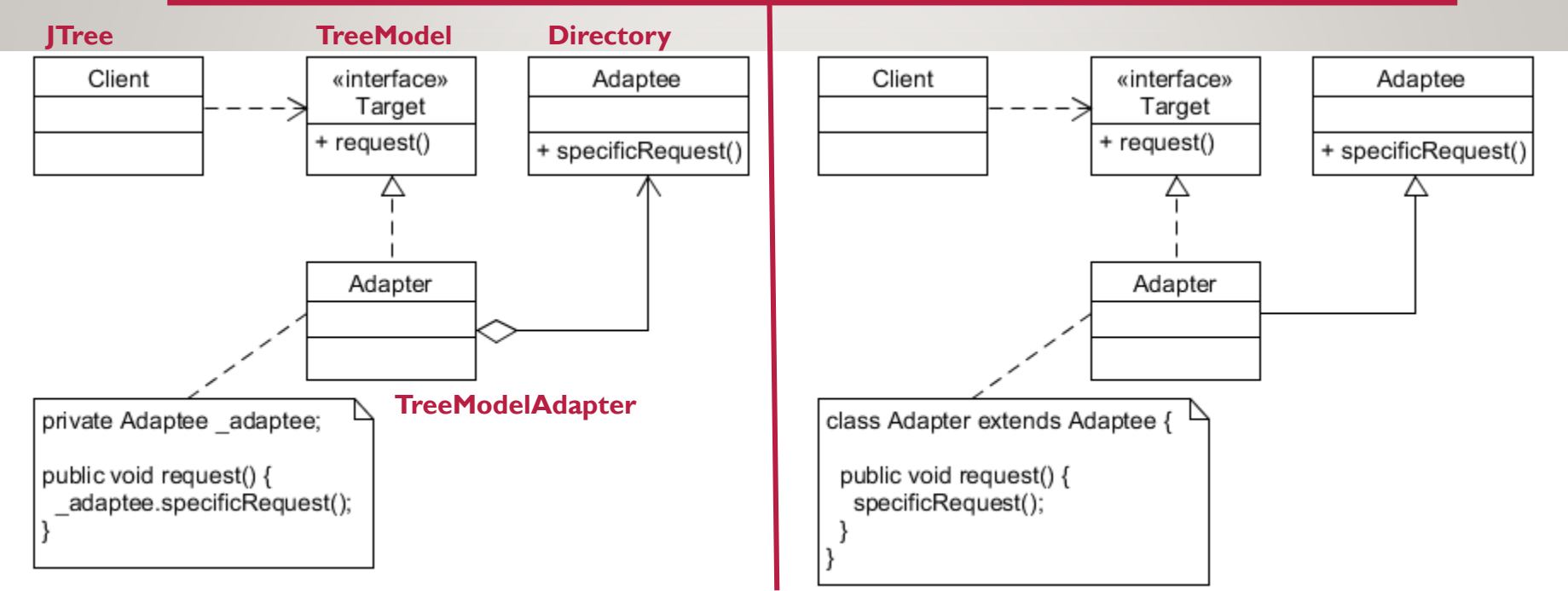
General Solution:



APPLICATION

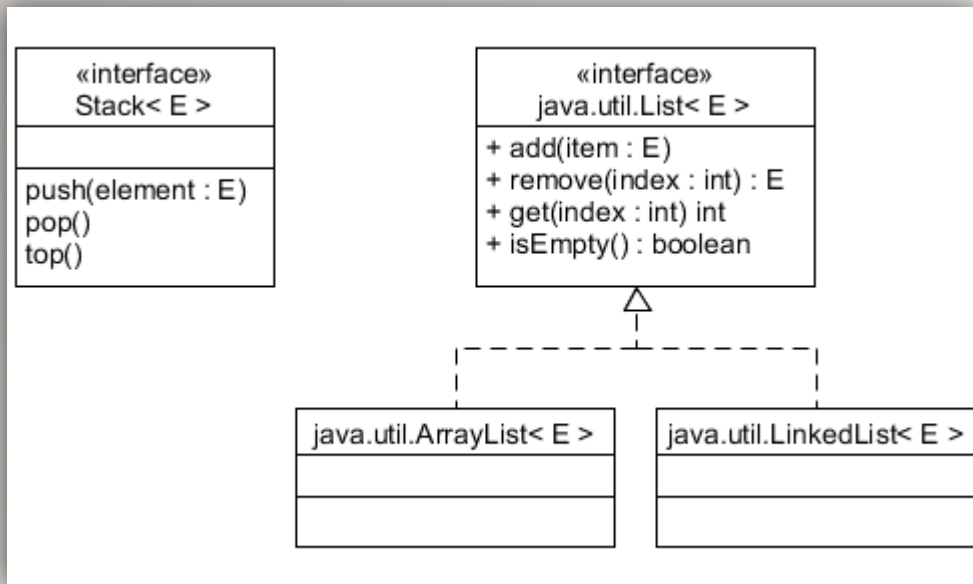
- Also known as
 - Wrapper
- Applicability
 - Use the Adapter pattern when
 - You want to use an existing class, and its interface does not match the one you need
 - You want to create a reusable class that cooperates with unrelated or unforeseen classes
- Design objectives satisfied:
 - *flexibility, reusability, robustness*

STRUCTURE



Containment	Inheritance
Simplicity. A class user need only look at a single class definition (the adapter class) to see what messages can be sent to an instance of the class.	A subclass's methods are generally a superset of those of its superclasses. A subclass (adapter) user, in general, needs to study many class definitions.
The adapter class must provide an implementation for all required methods.	Brevity. Subclass definitions tend to be shorter since required functionality is often inherited without change.
The class being reused can be fully encapsulated, and simply serves as a hidden implementation detail. An instance of the new class needn't expose its contained instance.	Possible inappropriate behaviour. All public inherited methods can be called on a subclass instance – be they appropriate or not.
There is no relationship between the adapter class and the class being reused. An instance of one can't be substituted for an instance of the other.	Promotes reuse of polymorphic methods. An existing method that works with an instance of the superclass will work, without modification, with an instance of the subclass.
Ease of implementation change (compile-time). The new class (adapter) can be modified to reuse an alternative existing class without affecting users of the new class.	Modifying a class definition to change its superclass will almost certainly impact on users of the subclass.
Ease of implementation change (run-time). It is possible for an instance of the new class to replace its contained instance with an object of a different class at run-time.	Inheritance is a compile-time relationship; a subclass can't change its superclass at run-time.
Information hiding is more naturally enforced. The new class doesn't have privileged access to the class being reused.	Can compromise information hiding. Where a subclass has privileged (i.e. protected) access to superclass variables, any change of the superclass's state implementation may impact subclasses.

EXERCISE



- Consider an interface for a **Stack**, a **FIFO** data structure
 - In implementing the stack, you can reuse a **List** implementation
 - Can you implement an **adapter** for this problem?

REVIEW

- Learnt something about:
 - The role of Adapters in developing software where:
 - Classes are often required to work together despite having incompatible interfaces
 - APIs are designed to work with unforeseen classes via Adapters
 - Using Adapters helps to promote:
 - Reusability – existing classes are reused and not re-invented
 - Flexibility – APIs can be designed today to work with unknown classes of tomorrow
 - Robustness – existing classes can be reused without change

REFERENCES

➤ [Books]

- Chapter 6 – Design Patterns (**Canvas**), Object-Oriented Software Engineering: Practical Software Development using UML and Java by Timothy Lethbridge and Robert Laganieri
- Design Patterns: Elements of Reusable Object-Oriented Software by John Vlissides, Ralph Johnson, Richard Helm, Erich Gamma
- Agile Software Development, Principles, Patterns, and Practices by Robert C Martin

➤ [Media]

- <https://docs.oracle.com/javase/tutorial/uiswing/>
- Robert Martin, Design Principles and Design Patterns, https://fi.ort.edu.uy/innovaportal/file/2032/1/design_principles.pdf
- A JavaWorld introductory article on design patterns: <http://www.javaworld.com/javaworld/jw-10-2001/jw-1012-designpatterns.html>

➤ [See more on Canvas Lecture page]